Panorama des modèles de régression en Machine Learning (formules, explications, exemples, code Python)

Georf MIGUIAMA BAMBA

4 février 2026

Table des matières

1 Notions de base (régression supervisée) 3
1.1 Mesures d'évaluation courantes . . . 3
1.2 Gabarit Python commun (dataset + split + métriques) . . . 3

2 Régressions linéaires (paramétriques) 3
2.1 Moindres carrés ordinaires (OLS) / Linear Regression . . . 3
2.2 Régression polynomiale (features polynomiales + linéaire) . . . 4
2.3 Ridge (L2) . . . 4
2.4 Lasso (L1) . . . 4
2.5 Elastic Net (L1 + L2) . . . 4
2.6 LARS / LassoLars . . . 5
2.7 SGDRegressor (descente de gradient stochastique) . . . 5
2.8 Régressions robustes (outliers) . . . 5
2.8.1 HuberRegressor . . . 5
2.8.2 RANSACRegressor . . . 5
2.8.3 TheilSenRegressor . . . 6
2.9 Régressions bayésiennes linéaires . . . 6
2.9.1 BayesianRidge . . . 6
2.9.2 ARDRegression (Automatic Relevance Determination) . . . 6
2.10 Régression quantile (QuantileRegressor) . . . 6
2.11 Isotonic Regression (monotone 1D) . . . 7
2.12 PLS (Partial Least Squares Regression) . . . 7

3 Modèles linéaires généralisés (GLM) pour cibles non gaussiennes 7
3.1 PoissonRegressor (comptages) . . . 7
3.2 GammaRegressor (positif continu) . . . 7
3.3 TweedieRegressor (famille flexible) . . . 8

4 Méthodes à noyaux / marges 8
4.1 SVR (Support Vector Regression) . . . 8
4.2 LinearSVR (approx plus scalable) . . . 8
4.3 Kernel Ridge Regression (KRR) . . . 8

5 Processus gaussiens (Bayésien non-paramétrique) 9
5.1 GaussianProcessRegressor . . . 9

6 Méthodes à base de voisins 9
6.1 KNN Regressor . . . 9

7 Arbres de décision et forêts **9**
7.1 DecisionTreeRegressor . . . 9
7.2 ExtraTreeRegressor (arbres extrêmement randomisés) . . . 10
7.3 RandomForestRegressor (bagging d'arbres) . . . 10
7.4 ExtraTreesRegressor . . . 10

8 Boosting (arbres additifs) **10**
8.1 GradientBoostingRegressor . . . 10
8.2 HistGradientBoostingRegressor (rapide, grands datasets) . . . 10
8.3 AdaBoostRegressor . . . 10

9 Bagging, voting, stacking (méta-modèles) **11**
9.1 BaggingRegressor . . . 11
9.2 VotingRegressor . . . 11
9.3 StackingRegressor . . . 11

10 Réseaux de neurones **12**
10.1 MLPRegressor (scikit-learn) . . . 12
10.2 Exemple Keras (réseau profond) . . . 12

11 Splines et modèles additifs **12**
11.1 Spline regression (SplineTransformer + modèle linéaire) . . . 12
11.2 GAM (Generalized Additive Models) via pyGAM . . . 13

12 Régression multi-sorties et multi-tâches **13**
12.1 MultiOutputRegressor (wrapper) . . . 13
12.2 MultiTaskLasso / MultiTaskElasticNet . . . 13

13 Gradient boosting moderne (libs externes incontournables) **13**
13.1 XGBoost . . . 14
13.2 LightGBM . . . 14
13.3 CatBoost . . . 14

14 Résumé par familles (aide-mémoire) **14**

15 Remarque sur "tous les modèles" **15**

1 Notions de base (régression supervisée)

On cherche une fonction $f : \mathbb{R}^d \to \mathbb{R}$ (ou $\mathbb{R}^m$ en multi-sorties) qui prédit y à partir de x en minimisant une perte sur un jeu d'entraînement $\{(x_i, y_i)\}_{i=1}^n$:

$$\min_{f \in \mathcal{F}} \sum_{i=1}^{n} \ell\big(y_i, f(x_i)\big) + \lambda\, \Omega(f)$$

où Ω est une régularisation (complexité) et $\lambda \geq 0$.

1.1 Mesures d'évaluation courantes

- MSE : $\frac{1}{n}\sum(y-\hat{y})^2$
- RMSE : $\sqrt{\text{MSE}}$
- MAE : $\frac{1}{n}\sum|y-\hat{y}|$
- R^2 : $1 - \frac{\sum(y-\hat{y})^2}{\sum(y-\bar{y})^2}$

1.2 Gabarit Python commun (dataset + split + métriques)

Exemple avec `California Housing` (régression continue).

```
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error,
    r2_score

X, y = fetch_california_housing(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

def report(model, X_test=X_test, y_test=y_test):
    y_pred = model.predict(X_test)
    rmse = mean_squared_error(y_test, y_pred, squared=False)
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    return {"RMSE": rmse, "MAE": mae, "R2": r2}
```

2 Régressions linéaires (paramétriques)

2.1 Moindres carrés ordinaires (OLS) / Linear Regression

Idée : modèle linéaire $\hat{y} = w^\top x + b$ en minimisant la MSE :

$$\min_{w,b} \sum_{i=1}^{n} (y_i - (w^\top x_i + b))^2$$

Quand : baseline, interprétable, rapide, sensible aux outliers et à la colinéarité.

```
from sklearn.linear_model import LinearRegression

model = LinearRegression().fit(X_train, y_train)
print(report(model))
```

2.2 Régression polynomiale (features polynomiales + linéaire)

Idée : transformer x en $\phi(x)$ (monômes) puis OLS/Ridge/Lasso.

$$\hat{y} = w^\top \phi(x)$$

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Ridge

model = make_pipeline(
    PolynomialFeatures(degree=2, include_bias=False),
    Ridge(alpha=1.0)
).fit(X_train, y_train)
print(report(model))
```

2.3 Ridge (L2)

$$\min_w \sum (y_i - w^\top x_i)^2 + \lambda \|w\|_2^2$$

Stabilise en présence de colinéarité ; garde tous les coefficients.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge

model = make_pipeline(StandardScaler(), Ridge(alpha=1.0)).fit(X_train,
    y_train)
print(report(model))
```

2.4 Lasso (L1)

$$\min_w \sum (y_i - w^\top x_i)^2 + \lambda \|w\|_1$$

Favorise la sparsité (sélection de variables), peut être instable si variables corrélées.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Lasso

model = make_pipeline(StandardScaler(), Lasso(alpha=0.001, max_iter=10000)
    ).fit(X_train, y_train)
print(report(model))
```

2.5 Elastic Net (L1 + L2)

$$\min_w \sum (y_i - w^\top x_i)^2 + \lambda \Big(\alpha \|w\|_1 + \tfrac{1-\alpha}{2} \|w\|_2^2 \Big)$$

Compromis Lasso/Ridge.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import ElasticNet
```

```
model = make_pipeline(StandardScaler(), ElasticNet(alpha=0.001, l1_ratio
    =0.5, max_iter=10000)) \
        .fit(X_train, y_train)
print(report(model))
```

2.6 LARS / LassoLars

Algorithmes efficaces quand d est grand et solution sparse recherchée.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LassoLars

model = make_pipeline(StandardScaler(), LassoLars(alpha=0.001)).fit(
    X_train, y_train)
print(report(model))
```

2.7 SGDRegressor (descente de gradient stochastique)

Optimise des pertes linéaires (MSE, Huber, epsilon-insensitive...) en mini-batch.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import SGDRegressor

model = make_pipeline(
    StandardScaler(),
    SGDRegressor(loss="squared_error", alpha=1e-4, max_iter=5000, tol=1e
        -3, random_state=42)
).fit(X_train, y_train)
print(report(model))
```

2.8 Régressions robustes (outliers)

2.8.1 HuberRegressor

Perte quadratique près de 0, linéaire pour grandes erreurs (robuste) :

$$\ell_\delta(r) = \begin{cases} \frac{1}{2}r^2 & |r| \leq \delta \\ \delta(|r| - \frac{1}{2}\delta) & |r| > \delta \end{cases}$$

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import HuberRegressor

model = make_pipeline(StandardScaler(), HuberRegressor()).fit(X_train,
    y_train)
print(report(model))
```

2.8.2 RANSACRegressor

Ajuste sur sous-échantillons (inliers) pour ignorer outliers.

```
from sklearn.linear_model import RANSACRegressor, LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
```

```
base = LinearRegression()
model = make_pipeline(StandardScaler(), RANSACRegressor(estimator=base,
    random_state=42)) \
        .fit(X_train, y_train)
print(report(model))
```

2.8.3 TheilSenRegressor

Robuste, coûteux si grand n.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import TheilSenRegressor

model = make_pipeline(StandardScaler(), TheilSenRegressor(random_state=42)
    ) \
        .fit(X_train, y_train)
print(report(model))
```

2.9 Régressions bayésiennes linéaires

2.9.1 BayesianRidge

Priors gaussiens sur w et estimation bayésienne (régularisation automatique).

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import BayesianRidge

model = make_pipeline(StandardScaler(), BayesianRidge()).fit(X_train,
    y_train)
print(report(model))
```

2.9.2 ARDRegression (Automatic Relevance Determination)

Pousse certains poids vers 0 (sélection bayésienne).

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import ARDRegression

model = make_pipeline(StandardScaler(), ARDRegression()).fit(X_train,
    y_train)
print(report(model))
```

2.10 Régression quantile (QuantileRegressor)

Prédit un quantile conditionnel via la perte *pinball* :

$$\ell_\tau(r) = \max(\tau r, (\tau - 1)r)$$

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import QuantileRegressor

model = make_pipeline(StandardScaler(), QuantileRegressor(quantile=0.5,
    alpha=0.0, solver="highs")) \
        .fit(X_train, y_train)
print(report(model))
```

2.11 Isotonic Regression (monotone 1D)

Si la relation est monotone en 1 dimension (ou après projection).

```
import numpy as np
from sklearn.isotonic import IsotonicRegression
from sklearn.model_selection import train_test_split

# Exemple 1D synth tique (l'isotonic est 1D)
rng = np.random.RandomState(42)
x = np.linspace(0, 10, 200)
y = 0.5*x + rng.normal(scale=1.0, size=x.shape)

xtr, xte, ytr, yte = train_test_split(x, y, test_size=0.2, random_state
    =42)

iso = IsotonicRegression(increasing=True).fit(xtr, ytr)
pred = iso.predict(xte)
```

2.12 PLS (Partial Least Squares Regression)

Utile quand d grand et variables très corrélées (réduction supervisée).

```
from sklearn.cross_decomposition import PLSRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

model = make_pipeline(StandardScaler(), PLSRegression(n_components=5)).fit
    (X_train, y_train)
# PLSRegression renvoie (n,1); scikit g re souvent, sinon .ravel()
print(report(model))
```

3 Modèles linéaires généralisés (GLM) pour cibles non gaussiennes

Si y est positive/compte, on choisit une distribution et un lien.

3.1 PoissonRegressor (comptages)

$$y \sim \text{Poisson}(\mu), \quad \log \mu = w^\top x$$

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import PoissonRegressor

# Attention: y doit  tre  >= 0 (comptages)
# Ici CaliforniaHousing n'est pas adapt ; exemple d'API seulement:
model = make_pipeline(StandardScaler(), PoissonRegressor(alpha=1.0,
    max_iter=1000))
```

3.2 GammaRegressor (positif continu)

$$y > 0, \quad \log \mu = w^\top x$$

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import GammaRegressor

model = make_pipeline(StandardScaler(), GammaRegressor(alpha=1.0, max_iter
    =1000))
```

3.3 TweedieRegressor (famille flexible)

Inclut Poisson/Gamma (selon `power`).

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import TweedieRegressor

model = make_pipeline(StandardScaler(), TweedieRegressor(power=1.5, alpha
    =1.0, link="log"))
```

4 Méthodes à noyaux / marges

4.1 SVR (Support Vector Regression)

Epsilon-insensitive loss, kernel possible (RBF, poly...).

$$\min \frac{1}{2}\|w\|^2 + C\sum(\xi_i + \xi_i^)$$

sous contraintes $|y_i - f(x_i)| \leq \epsilon + \text{slack}$.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR

model = make_pipeline(StandardScaler(), SVR(kernel="rbf", C=10.0, epsilon
    =0.1)) \
        .fit(X_train, y_train)
print(report(model))
```

4.2 LinearSVR (approx plus scalable)

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVR

model = make_pipeline(StandardScaler(), LinearSVR(C=1.0, epsilon=0.1,
    random_state=42)) \
        .fit(X_train, y_train)
print(report(model))
```

4.3 Kernel Ridge Regression (KRR)

Ridge dans un espace à noyau :

$$\min_{f\in\mathcal{H}} \sum(y_i - f(x_i))^2 + \lambda\|f\|_{\mathcal{H}}^2$$

```
from sklearn.kernel_ridge import KernelRidge
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

model = make_pipeline(
    StandardScaler(),
    KernelRidge(alpha=1.0, kernel="rbf", gamma=0.1)
).fit(X_train, y_train)
print(report(model))
```

5 Processus gaussiens (Bayésien non-paramétrique)

5.1 GaussianProcessRegressor

Priors sur les fonctions $f \sim \mathcal{GP}(m(x), k(x, x'))$, donne une moyenne et une incertitude.

```
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, ConstantKernel as C
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

kernel = C(1.0) * RBF(length_scale=1.0)
gpr = GaussianProcessRegressor(kernel=kernel, alpha=1e-2, normalize_y=True
    , random_state=42)

model = make_pipeline(StandardScaler(), gpr).fit(X_train[:2000], y_train
    [:2000])  # souvent co teux
print(report(model, X_test=X_test[:2000], y_test=y_test[:2000]))
```

6 Méthodes à base de voisins

6.1 KNN Regressor

Prédiction par moyenne pondérée des k plus proches voisins.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsRegressor

model = make_pipeline(StandardScaler(), KNeighborsRegressor(n_neighbors
    =15, weights="distance")) \
        .fit(X_train, y_train)
print(report(model))
```

7 Arbres de décision et forêts

7.1 DecisionTreeRegressor

Partitionne l'espace ; minimise MSE (ou MAE) par noeud.

```
from sklearn.tree import DecisionTreeRegressor

model = DecisionTreeRegressor(max_depth=10, random_state=42).fit(X_train,
    y_train)
print(report(model))
```

7.2 ExtraTreeRegressor (arbres extrêmement randomisés)

```
from sklearn.tree import ExtraTreeRegressor

model = ExtraTreeRegressor(max_depth=20, random_state=42).fit(X_train,
    y_train)
print(report(model))
```

7.3 RandomForestRegressor (bagging d'arbres)

```
from sklearn.ensemble import RandomForestRegressor

model = RandomForestRegressor(
    n_estimators=300, max_depth=None, min_samples_leaf=1, random_state=42,
        n_jobs=-1
).fit(X_train, y_train)
print(report(model))
```

7.4 ExtraTreesRegressor

Plus de randomisation ; souvent performant et rapide.

```
from sklearn.ensemble import ExtraTreesRegressor

model = ExtraTreesRegressor(n_estimators=500, random_state=42, n_jobs=-1).
    fit(X_train, y_train)
print(report(model))
```

8 Boosting (arbres additifs)

8.1 GradientBoostingRegressor

```
from sklearn.ensemble import GradientBoostingRegressor

model = GradientBoostingRegressor(random_state=42).fit(X_train, y_train)
print(report(model))
```

8.2 HistGradientBoostingRegressor (rapide, grands datasets)

```
from sklearn.ensemble import HistGradientBoostingRegressor

model = HistGradientBoostingRegressor(random_state=42).fit(X_train,
    y_train)
print(report(model))
```

8.3 AdaBoostRegressor

```
from sklearn.ensemble import AdaBoostRegressor

model = AdaBoostRegressor(random_state=42, n_estimators=300, learning_rate
    =0.05) \
        .fit(X_train, y_train)
print(report(model))
```

9 Bagging, voting, stacking (méta-modèles)

9.1 BaggingRegressor

```
from sklearn.ensemble import BaggingRegressor
from sklearn.tree import DecisionTreeRegressor

model = BaggingRegressor(
    estimator=DecisionTreeRegressor(random_state=42),
    n_estimators=200,
    random_state=42,
    n_jobs=-1
).fit(X_train, y_train)
print(report(model))
```

9.2 VotingRegressor

Moyenne (souvent) de plusieurs régressseurs.

```
from sklearn.ensemble import VotingRegressor
from sklearn.linear_model import Ridge
from sklearn.ensemble import RandomForestRegressor
from sklearn.svm import SVR
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

r1 = make_pipeline(StandardScaler(), Ridge(alpha=1.0))
r2 = RandomForestRegressor(n_estimators=300, random_state=42, n_jobs=-1)
r3 = make_pipeline(StandardScaler(), SVR(C=10.0, gamma="scale"))

model = VotingRegressor([("ridge", r1), ("rf", r2), ("svr", r3)]).fit(
   X_train, y_train)
print(report(model))
```

9.3 StackingRegressor

Apprend un méta-modèle sur les prédictions des base-models.

```
from sklearn.ensemble import StackingRegressor
from sklearn.linear_model import RidgeCV
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR

estimators = [
    ("svr", make_pipeline(StandardScaler(), SVR(C=10.0))),
    ("hgb", HistGradientBoostingRegressor(random_state=42)),
]

model = StackingRegressor(
    estimators=estimators,
    final_estimator=RidgeCV(alphas=(0.1, 1.0, 10.0))
).fit(X_train, y_train)
print(report(model))
```

10 Réseaux de neurones

10.1 MLPRegressor (scikit-learn)

Perceptron multi-couches ; scaling recommandé.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPRegressor

model = make_pipeline(
    StandardScaler(),
    MLPRegressor(hidden_layer_sizes=(128, 64), activation="relu",
                 alpha=1e-4, max_iter=200, random_state=42)
).fit(X_train, y_train)
print(report(model))
```

10.2 Exemple Keras (réseau profond)

```
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.datasets import fetch_california_housing
import tensorflow as tf

X, y = fetch_california_housing(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
   random_state=42)

scaler = StandardScaler().fit(X_train)
X_train_s = scaler.transform(X_train)
X_test_s  = scaler.transform(X_test)

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(X_train_s.shape[1],)),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(1)
])

model.compile(optimizer=tf.keras.optimizers.Adam(1e-3), loss="mse")
model.fit(X_train_s, y_train, validation_split=0.1, epochs=20, batch_size
   =256, verbose=0)

y_pred = model.predict(X_test_s).ravel()
```

11 Splines et modèles additifs

11.1 Spline regression (SplineTransformer + modèle linéaire)

Approxime des non-linéarités de manière contrôlée.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import SplineTransformer, StandardScaler
from sklearn.linear_model import Ridge

model = make_pipeline(
    StandardScaler(),
```

```
    SplineTransformer(n_knots=8, degree=3, include_bias=False),
    Ridge(alpha=1.0)
).fit(X_train, y_train)
print(report(model))
```

11.2 GAM (Generalized Additive Models) via pyGAM

Forme additive : $\hat{y} = \beta_0 + \sum_j f_j(x_j)$. (`pyGAM` est externe à scikit-learn)

```
# pip install pygam
from pygam import LinearGAM, s

gam = LinearGAM(sum([s(i) for i in range(X_train.shape[1])])).fit(X_train,
    y_train)
y_pred = gam.predict(X_test)
```

12 Régression multi-sorties et multi-tâches

12.1 MultiOutputRegressor (wrapper)

Permet d'utiliser un régressseur pour $y \in \mathbb{R}^m$.

```
import numpy as np
from sklearn.multioutput import MultiOutputRegressor
from sklearn.ensemble import RandomForestRegressor

# Exemple: fabriquer une cible multi-sorties
Y = np.vstack([y, 2*y + 0.1*np.random.randn(*y.shape)]).T
X_train2, X_test2, Y_train2, Y_test2 = train_test_split(X, Y, test_size
   =0.2, random_state=42)

base = RandomForestRegressor(n_estimators=200, random_state=42, n_jobs=-1)
model = MultiOutputRegressor(base).fit(X_train2, Y_train2)
pred = model.predict(X_test2)
```

12.2 MultiTaskLasso / MultiTaskElasticNet

Couplent les sorties et sélectionnent des features communes.

```
from sklearn.linear_model import MultiTaskElasticNet
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

model = make_pipeline(StandardScaler(), MultiTaskElasticNet(alpha=0.01,
   l1_ratio=0.5, max_iter=10000)) \
        .fit(X_train2, Y_train2)
pred = model.predict(X_test2)
```

13 Gradient boosting moderne (libs externes incontournables)

Ces modèles dominent souvent sur données tabulaires.

13.1 XGBoost

```
# pip install xgboost
from xgboost import XGBRegressor

model = XGBRegressor(
    n_estimators=2000, learning_rate=0.03,
    max_depth=6, subsample=0.8, colsample_bytree=0.8,
    reg_alpha=0.0, reg_lambda=1.0,
    random_state=42, n_jobs=-1
).fit(X_train, y_train)

print(report(model))
```

13.2 LightGBM

```
# pip install lightgbm
from lightgbm import LGBMRegressor

model = LGBMRegressor(
    n_estimators=5000, learning_rate=0.02,
    num_leaves=64, subsample=0.8, colsample_bytree=0.8,
    random_state=42, n_jobs=-1
).fit(X_train, y_train)

print(report(model))
```

13.3 CatBoost

```
# pip install catboost
from catboost import CatBoostRegressor

model = CatBoostRegressor(
    depth=8, learning_rate=0.05, n_estimators=3000,
    loss_function="RMSE", random_seed=42, verbose=False
).fit(X_train, y_train)

print(report(model))
```

14 Résumé par familles (aide-mémoire)

Famille	Modèles typiques
Linéaires	OLS, Ridge, Lasso, ElasticNet, SGDRegressor, PLS
Robustes	Huber, RANSAC, Theil-Sen
Quantiles	QuantileRegressor
GLM	Poisson, Gamma, Tweedie
Noyaux	SVR, Kernel Ridge
Bayésien non-param	Gaussian Processes
Voisins	KNN Regressor
Arbres/Forêts	DecisionTree, RandomForest, ExtraTrees
Boosting	GBDT, HistGBDT, AdaBoost, XGBoost, LightGBM, CatBoost
Neural nets	MLPRegressor, Keras/PyTorch
Splines/GAM	Splines + linéaire, GAM (pyGAM)
Méta-modèles	Bagging, Voting, Stacking

15 Remarque sur “tous les modèles”

Il n’existe pas de liste exhaustive finie : on peut composer (features, pertes, régularisations, architectures), et il existe des variantes spécialisées (régression contrainte, monotone multi-dim, censurée, survival, hiérarchique bayésienne, etc.). Le présent document couvre les familles ML standard les plus utilisées et celles disponibles dans les bibliothèques majeures.